FAST National University of Computing & Emerging Sciences

Department of Computer Science

CHAPTER 1 – INTRODUCTION

Operating System Concepts

Quiz Preparation Notes

Name : Arslan Tariq

Roll No : 24P-0610

Section : 4A

Textbook : Silberschatz, 10th Edition

Contents

1 What Operating Systems Do

✎ Definition

An **Operating System (OS)** is software that manages computer hardware, provides an environment for application programs to run, and acts as an intermediary between the user and hardware.

1.1 Four Components of a Computer System

1. **Hardware** – CPU, memory, I/O devices (provides basic computing resources)
2. **Operating System** – controls hardware, coordinates its use among applications
3. **Application Programs** – word processors, compilers, browsers, games
4. **Users** – people, machines, other computers

1.2 OS Viewed from Different Perspectives

- **User View:** Ease of use, performance, usability (not resource utilization)
- **System View:** OS is a **resource allocator** – manages CPU, memory, I/O, storage
- **Control Program View:** OS is a **control program** – controls execution of user programs to prevent errors and improper use

★ Important for Quiz

There is **no universally accepted definition** of an OS. A common definition: "The one program running at all times on the computer is the **kernel**." Everything else is either a **system program** or an **application program**.

❖ Key Point

Middleware = a set of software frameworks that provide additional services to application developers (common in mobile OS like Android/iOS).

2 Computer-System Organization

2.1 Basic Operation

- One or more CPUs and device controllers connected through a common **bus** to shared **memory**
- Each device controller is in charge of a specific device type (disk, keyboard, display)
- CPU and device controllers can execute **in parallel**, competing for memory cycles
- A **memory controller** synchronizes access to shared memory

2.2 Interrupts

✎ Definition

An **interrupt** is a signal from hardware (or software) to the CPU indicating that an event needs immediate attention.

- Hardware triggers interrupt by sending signal to CPU via **system bus**
- Software triggers interrupt via **system call** (also called **trap** or **monitor call**)
- CPU transfers execution to **Interrupt Service Routine (ISR)** via the **interrupt vector** (table of addresses)
- Interrupted instruction's address is **saved** and **restored** after servicing
- **Trap/Exception** = software-generated interrupt (caused by error or user request)

★ Important for Quiz

Interrupt vs Trap:
• **Interrupt** = asynchronous, caused by **hardware** events
• **Trap** = synchronous, caused by **software** (errors, system calls)
• Yes, traps **can** be generated intentionally by user programs (e.g., to request OS services via system calls)

2.3 Storage Structure

- Programs must be loaded into **main memory (RAM)** to execute
- RAM is **volatile** – loses data when power is off
- **ROM / EEPROM / Firmware** = nonvolatile, stores bootstrap program
- **Bootstrap program** = first program to run at power-on; loads the OS kernel
- **Secondary storage** = nonvolatile extension of main memory (HDD, SSD)

2.4 Storage Hierarchy (Fastest to Slowest)

1. Registers
2. Cache
3. Main Memory (RAM)
4. Nonvolatile Memory (NVM/SSD)
5. Hard-Disk Drives (HDD)
6. Optical Disk
7. Magnetic Tapes

✔ Remember

Rule: Higher level = faster, smaller, more expensive, volatile.
Lower level = slower, larger, cheaper, nonvolatile.

2.5 I/O Structure

- **Device controller** maintains a local buffer and special-purpose registers
- **Device driver** = OS component that provides uniform interface to device controller
- **DMA (Direct Memory Access)** = device controller transfers blocks of data directly to/from memory **without CPU intervention**; CPU is interrupted only once per block (not per byte)

★ Important for Quiz

DMA Process:
1. CPU sets up DMA transfer (source, destination, count)
2. DMA controller transfers data directly to memory
3. DMA controller interrupts CPU when transfer is **complete**
4. CPU can execute other programs during transfer

Interference: DMA competes with CPU for memory bus cycles (cycle stealing)

3 Computer-System Architecture

3.1 Single-Processor Systems

- One general-purpose CPU with one processing core
- May have special-purpose processors (disk controller, keyboard controller)
- Now rare – most systems are multiprocessor

3.2 Multiprocessor Systems (Parallel Systems)

✎ Definition

Multiprocessor systems have two or more processors sharing the computer bus, clock, memory, and peripheral devices.

Three main advantages:

1. **Increased throughput** – more work done in less time
2. **Economy of scale** – shared resources cost less than separate systems
3. **Increased reliability** – failure of one processor doesn't halt the system

Two types:

- **Asymmetric Multiprocessing (AMP):** Boss-worker relationship; one master CPU controls the system, assigns tasks to worker CPUs
- **Symmetric Multiprocessing (SMP):** All processors are **peers**; each runs its own copy of the OS; all share memory. *Most common today.*

❖ Key Point

Multicore = multiple computing cores on a **single chip**. Faster communication (on-chip) and less power than multiple separate chips. Each core has its own **local cache** (L1), and cores share a **common cache** (L2/L3).

★ Important for Quiz

Why two levels of cache in SMP?
• **L1 (local to each core)** – very fast, small, avoids bus traffic
• **L2/L3 (shared)** – larger, slower but reduces main memory access
This design balances **speed** and **data consistency**.

3.3 Clustered Systems

- Multiple **individual systems** (nodes) joined together via LAN or faster interconnect
- Provide **high availability** – if one node fails, others continue service
- Share **storage** via a SAN (Storage-Area Network)
- **Asymmetric clustering:** One machine on hot-standby, monitoring the active server
- **Symmetric clustering:** All nodes run applications and monitor each other

✔ Remember

Cluster vs Multiprocessor:
• **Multiprocessor** = multiple CPUs in **one system**, share memory
• **Cluster** = multiple **separate systems** networked together, each with own memory
• For high availability, cluster software monitors nodes and can **failover** (take over) if one node fails

4 Operating-System Operations

4.1 Multiprogramming and Multitasking

✎ Definition

Multiprogramming organizes jobs so that the CPU **always has one to execute**. Multiple jobs are kept in memory simultaneously. When one job waits for I/O, the OS switches to another.

✎ Definition

Multitasking (Time-sharing) is an extension of multiprogramming where the CPU switches between processes **so frequently** that users can interact with each program while it is running. Provides **fast response time**.

- Requires **CPU scheduling** – deciding which process runs next
- If processes don't fit in memory → **swapping** (moving processes in/out of memory)
- **Virtual memory** allows execution of processes not completely in memory

4.2 Dual-Mode Operation

★ Important for Quiz

Two modes of operation:
1. **User Mode** – executing user applications (restricted)
2. **Kernel Mode** (Supervisor/System/Privileged Mode) – executing OS code (full access)

A **mode bit** in hardware indicates the current mode: 0 = kernel, 1 = user.

System call switches from user mode to kernel mode. Return switches back to user mode.

Privileged instructions can execute **only in kernel mode:**

- Switch to kernel mode
- I/O control
- Timer management
- Interrupt management
- Clear memory
- Modify entries in device-status table
- Turn off interrupts

❖ Key Point

The dual-mode mechanism is a **rudimentary form of protection**: user programs cannot directly execute privileged instructions. Any attempt in user mode causes a **trap** to the OS, which terminates the offending program.

4.3 Timer

- Prevents a process from running forever (infinite loop)
- OS sets a **counter** that is decremented by the hardware clock
- When counter reaches zero → interrupt → OS regains control
- Setting the timer is a **privileged instruction**
- Can also compute current time by counting ticks from a known starting point

★ Important for Quiz

Linux: HZ and jiffies
- `HZ` = number of timer interrupts per second (e.g., 250)
- `jiffies` = number of timer interrupts since boot
- Seconds since boot = `jiffies / HZ`

5 Resource Management

5.1 Process Management

✎ Definition

A **process** is a **program in execution**. It is the **fundamental unit of work** in an OS. A program is a **passive** entity (file on disk); a process is an **active** entity.

- A process needs resources: CPU time, memory, files, I/O devices
- **Single-threaded** process has one **program counter** (PC)
- **Multi-threaded** process has one PC **per thread**
- OS responsibilities: create/delete processes, schedule, synchronize, communicate, handle deadlocks

5.2 Memory Management

- Main memory is a large array of bytes, each with its own address
- OS tracks which parts of memory are in use and by whom
- Decides which processes/data to move in/out of memory
- Allocates and deallocates memory space

5.3 File-System Management

- OS provides uniform, logical view of storage
- **File** = logical storage unit; **Directory** = organizes files
- OS responsibilities: create/delete files and directories, mapping files to storage, backup

5.4 Mass-Storage Management

- Programs stored on secondary storage (disk) until loaded into memory
- OS manages: free-space management, storage allocation, disk scheduling
- **Tertiary storage:** optical disks, magnetic tapes (for backups)

5.5 Cache Management

❖ Key Point

Caching = copying frequently used data into faster storage.

Two reasons caches are useful:
1. **Speed** – bridge the gap between fast CPU and slow memory/disk
2. **Reduce access time** – frequently accessed data served from cache

Problems:
• **Cache coherency** – multiple copies of same data must be consistent (critical in multiprocessor and distributed systems)
• **Limited size** – cache management must decide what to keep/replace
• Cannot make cache as large as disk because: cache uses **expensive, volatile** technology (SRAM); disk uses **cheap, nonvolatile** technology

5.6 I/O System Management

- OS hides hardware peculiarities from users
- I/O subsystem consists of: memory management (buffering, caching, spooling), device-driver interface, drivers for specific hardware

6 Security and Protection

✎ Definition

Protection = mechanism for controlling access of processes and users to resources defined by the OS.

Security = defense of the system against internal and external attacks (viruses, worms, denial-of-service, identity theft, etc.).

- Each user has a **User ID (UID)** and belongs to **Group ID (GID)**
- **Privilege escalation** allows users to gain higher permissions temporarily (e.g., `sudo` in UNIX)

✔ Remember

Memory Protection:
• Use **base register** and **limit register**
• Base = starting address of process's memory
• Limit = size of the range
• Any access outside [base, base+limit) → trap to OS
• Only OS (kernel mode) can modify base/limit registers

7 Virtualization

✎ Definition

Virtualization = technology that abstracts hardware of a single computer into several different execution environments, creating the illusion that each is running on its own private computer.

- **VMM (Virtual Machine Manager) / Hypervisor** = manages virtual machines
- **Type 0:** Hardware-based (firmware level)
- **Type 1:** OS-like software (VMware ESXi, runs directly on hardware)
- **Type 2:** Application-level (VirtualBox, VMware Workstation – runs on top of a host OS)
- **Emulation:** Simulates hardware of a different architecture
- Each VM is a **guest** running its own OS on the **host** system

8 Distributed Systems

✎ Definition

A **distributed system** is a collection of physically separate, networked computers that appear to users as a **single coherent system**.

- **Network** = communication path between two or more systems
- **LAN** = Local Area Network (within a building/campus)
- **WAN** = Wide Area Network (across cities/countries)
- **MAN** = Metropolitan Area Network (within a city)
- Network OS provides features like file sharing, remote login

8.1 Client–Server vs Peer-to-Peer

★ Important for Quiz

Client–Server Model:
- Server provides services (file, database, web)
- Client requests services
- Centralized management, easier security

Peer-to-Peer (P2P) Model:
- All nodes are **equals** – each can be client AND server
- No centralized server needed

- Scalable, no single point of failure
- Examples: BitTorrent, Skype, Bitcoin

P2P Advantages over Client–Server:
- No bottleneck at server, better scalability
- No single point of failure
- Resources distributed across all nodes

9 Kernel Data Structures

- **Lists:** Singly linked, doubly linked, circular linked list
- **Stacks:** LIFO (Last In, First Out) – used for function calls
- **Queues:** FIFO (First In, First Out) – used in CPU scheduling, print queues
- **Trees:** Binary tree, binary search tree, balanced BST – used for file systems, process hierarchies
- **Hash Functions & Maps:** Key → value mapping, used for fast lookups
- **Bitmaps:** String of n bits representing status of n items (0 = free, 1 = occupied) – used in disk/memory allocation

10 Computing Environments

- **Traditional Computing:** PCs, office networks, web-based terminals
- **Mobile Computing:** Smartphones, tablets (Android, iOS); extra features like GPS, gyroscope
- **Client–Server:** Server fulfills requests from clients
- **Peer-to-Peer:** All participants are equal
- **Cloud Computing:** Delivers computing as a service over the Internet
  - **SaaS** – Software as a Service (Gmail, Office 365)
  - **PaaS** – Platform as a Service (Google App Engine)
  - **IaaS** – Infrastructure as a Service (AWS EC2, Azure)
- **Real-Time Embedded Systems:** Most prevalent; strict timing constraints; found in cars, robots, medical devices

★ Important for Quiz

Mobile OS Challenges vs Traditional PC:
- Limited memory and processing power
- Battery life constraints
- Smaller screen size

- Limited storage
- Security concerns (lost/stolen devices)
- Need to support GPS, accelerometer, gyroscope, etc.

11 Free and Open-Source Operating Systems

- **Open Source:** Source code available for study, modification, redistribution
- **Free Software:** Licensed for no-cost use, redistribution, modification (GNU GPL)
- **GNU/Linux:** Started by Linus Torvalds (1991); kernel + GNU tools; many distributions (Ubuntu, Fedora, Debian)
- **BSD UNIX:** Derived from AT&T UNIX; variants: FreeBSD, NetBSD, OpenBSD, DragonflyBSD
- **Darwin:** Core kernel of macOS; based on BSD; open-sourced by Apple
- **Solaris:** Sun Microsystems (now Oracle); partially open-sourced as OpenSolaris → Project Illumos
- **Version control:** git (Linux), Subversion (FreeBSD)
- **VirtualBox** and **Qemu** = free VMMs to run different OS images

❖ **Key Point**

Open-Source OS Advantages:
- Free to use, study, modify, redistribute
- Community-driven rapid bug fixes and improvements
- Great learning tools for students
- Diversity of choices (Linux, BSD, Solaris)

Disadvantages:
- Limited commercial support (for some)
- Steeper learning curve
- Hardware compatibility can be an issue
- Security vulnerabilities are visible to attackers too

Exercise Answers

11.1 Practice Exercises (1.1 – 1.11)

Q 1.1: What are the three main purposes of an operating system?

1. **Convenience:** Makes the computer easier to use for users
2. **Efficiency:** Manages hardware resources efficiently (CPU, memory, I/O)
3. **Ability to evolve:** Should be constructed in a way that permits effective development, testing, and introduction of new system functions without interfering with existing services

Q 1.2: When is it appropriate for the OS to "waste" resources? Why is it not really wasteful?

Single-user systems (PCs, mobile devices) prioritize **user convenience and responsiveness** over resource utilization. For example, a GUI wastes CPU cycles on graphical rendering, but it maximizes **user productivity**. It is not truly wasteful because the goal is to optimize the **user's experience**, not the hardware's utilization. The resource being optimized is the **user's time**, which is more valuable.

Q 1.3: What is the main difficulty in writing an OS for a real-time environment?

The main difficulty is ensuring that the OS meets **strict timing constraints** (deadlines). The system must guarantee that critical tasks are processed within a **fixed time limit**. If the timing is not met, the system fails. The OS must minimize **interrupt latency** and ensure **deterministic** response times.

Q 1.4: Should the OS include applications like web browsers and mail programs?

Arguments FOR including:

- Better integration with the OS, improved user experience

- Guaranteed compatibility and optimized performance
- Convenience for users – system is ready to use out of the box

Arguments AGAINST including:

- Bloats the OS, increases its size and complexity
- Security vulnerabilities in applications can compromise the entire OS
- Anti-competitive behavior (as in the Microsoft antitrust case)
- Users should have freedom to choose their preferred applications

Q 1.5: How does the distinction between kernel mode and user mode function as a rudimentary form of protection?

- **Privileged instructions** (I/O operations, memory management, interrupt handling) can **only execute in kernel mode**
- User programs run in **user mode** and **cannot** directly execute privileged instructions
- If a user program attempts a privileged instruction, the hardware **traps** to the OS, which can terminate the offending program
- This prevents user programs from **interfering** with the OS or other programs

Q 1.6: Which of the following instructions should be privileged?

a. Set value of timer	→	**Privileged ✓**
b. Read the clock	→	Non-privileged
c. Clear memory	→	**Privileged ✓**
d. Issue a trap instruction	→	Non-privileged
e. Turn off interrupts	→	**Privileged ✓**
f. Modify entries in device-status table	→	**Privileged ✓**
g. Switch from user to kernel mode	→	**Privileged ✓**
h. Access I/O device	→	**Privileged ✓**

Q 1.7: OS in unmodifiable memory partition – what are two difficulties?

1. **Cannot update the OS:** Bug fixes, security patches, and new features cannot be applied since the OS code cannot be modified
2. **Cannot adapt to hardware changes:** New device drivers or configuration changes are impossible. The OS becomes rigid and unable to evolve with changing

requirements

Q 1.8: CPUs with more than two modes – two possible uses?

1. **Virtual Machine Monitor (VMM) mode:** A separate mode for the hypervisor, which has more privileges than user mode but fewer than kernel mode. This allows the VMM to control VMs without full kernel privileges
2. **Different privilege levels for system services:** System services (like device drivers) can run in an intermediate mode with limited privileges, reducing the damage from a buggy driver without giving it full kernel access

Q 1.9: How can timers compute the current time?

- A timer generates interrupts at a **known fixed interval** (e.g., every 1 ms)
- The OS maintains a **counter** (e.g., `jiffies` in Linux) that increments with each tick
- Starting from a **known reference time** (e.g., boot time or epoch), multiply the counter by the tick interval to get elapsed time
- Current time = reference time + (counter × tick interval)

Q 1.10: Give two reasons why caches are useful. Problems they solve? Problems they cause?

Two reasons caches are useful:

1. **Speed:** Provide faster access to frequently used data than slower main memory or disk
2. **Bridge speed gap:** CPU is much faster than main memory; cache reduces the CPU's wait time

Problems they cause:

- **Cache coherency:** In multiprocessor systems, multiple caches may hold different copies of the same data – keeping them consistent is complex
- **Limited size:** Cache is expensive (SRAM); replacement policies needed

Why not make cache as large as disk?

- Cache uses **expensive, volatile** SRAM; disk is **cheap, nonvolatile**
- Even if affordable, cache is volatile – data lost on power failure
- Disk provides **permanent** storage, cache does not

Q 1.11: Distinguish between client–server and peer-to-peer models.

Client–Server	Peer-to-Peer
Centralized server provides services	All nodes are equals (peers)
Clients request, server responds	Each node can request AND provide services
Single point of failure (server)	No single point of failure
Easier to manage and secure	More scalable, harder to manage
Server can become bottleneck	Load distributed across all peers
Examples: Web, email, databases	Examples: BitTorrent, Skype

11.2 Chapter Exercises (1.12 – 1.27)

Q 1.12: How do clustered systems differ from multiprocessor systems? Requirements for high availability?

Differences:

- **Multiprocessor:** Multiple CPUs in a **single system**, sharing memory and bus
- **Clustered:** Multiple **independent systems** connected via network, each with own CPU and memory

For high availability:

- Both machines must be networked and able to **monitor** each other
- Must have shared or replicated **storage** (SAN)
- **Failover** capability: if one fails, the other takes over its workload
- Cluster management software must coordinate access and detect failures

Q 1.13: Two ways cluster software can manage database access on shared disk?

1. Asymmetric Clustering:

- One node is active (processes all requests), the other is on hot-standby
- **Advantage:** Simpler, no data consistency issues
- **Disadvantage:** Standby node wastes resources; not fully utilized

2. Symmetric Clustering (Parallel):

- Both nodes are active, both access the database simultaneously
- **Advantage:** Better resource utilization, higher throughput
- **Disadvantage:** Requires a **Distributed Lock Manager (DLM)** to handle concurrent access and prevent data corruption; more complex

Q 1.14: Purpose of interrupts? Interrupt vs trap? Can traps be generated intentionally?

Purpose: Interrupts alert the CPU that an event needs attention, allowing the CPU to respond to asynchronous events without continuously polling devices.

Interrupt vs Trap:

- **Interrupt: Hardware-generated**, asynchronous (e.g., keyboard press, disk completion)
- **Trap: Software-generated**, synchronous (e.g., division by zero, system call)

Can traps be intentional? Yes. User programs generate traps intentionally to invoke **system calls** – requesting OS services like file read/write, memory allocation, process creation.

Q 1.15: How can Linux HZ and jiffies determine uptime?

- `HZ` = frequency of timer interrupts per second (constant, e.g., 250)
- `jiffies` = total number of timer interrupts since system boot (counter)
- **Seconds since boot** = `jiffies / HZ`
- Example: If `HZ = 250` and `jiffies = 750000`, then uptime = 750000 / 250 = **3000 seconds**

Q 1.16: DMA – CPU interface, completion detection, interference?

a. How does CPU interface with device for DMA?

- CPU writes the source address, destination address, and byte count to the DMA controller's registers
- CPU signals the DMA controller to begin the transfer

b. How does CPU know when transfer is complete?

- The DMA controller sends an **interrupt** to the CPU when the entire block transfer is finished

c. Does DMA interfere with user programs?

- **Yes**, partially. DMA and CPU compete for the **memory bus** (called **cycle stealing**)
- During DMA transfers, the CPU may be **slowed down** because it cannot access memory while the DMA controller is using the bus
- However, the CPU can still execute instructions from its **cache** during DMA transfers

Q 1.17: Can a secure OS be built without hardware-privileged mode?

Argument that it IS possible:

- Protection can be achieved through **software interpretation** – the OS can interpret all user code (like Java's JVM), ensuring no unsafe operations
- Language-based protection or a **virtual machine layer** can enforce security

Argument that it is NOT possible:

- Without hardware mode distinction, a malicious program can execute **any instruction**, including privileged ones
- No hardware protection means software-only protection can always be **bypassed** by sufficiently clever code
- Performance overhead of software interpretation makes it impractical

Q 1.18: Why do SMP systems have different levels of caches?

- **L1 cache (local to each core):** Very small, very fast; provides each core its own private fast storage, reducing contention on the shared bus
- **L2/L3 cache (shared among cores):** Larger, slightly slower; shared data is accessible to all cores without going to slow main memory
- This design **balances speed and data sharing**: local caches give speed, shared caches give efficient data sharing
- Local caches reduce **bus traffic**; shared caches improve **hit rate** for shared data

Q 1.19: Rank storage systems from slowest to fastest.

Rank	Storage	Speed
6 (Slowest)	f. Magnetic tapes	Slowest
5	c. Optical disk	
4	a. Hard-disk drives	
3	e. Nonvolatile memory (SSD)	
2	d. Main memory (RAM)	
1	g. Cache	
0 (Fastest)	b. Registers	Fastest

Order: Magnetic tapes < Optical disk < HDD < NVM < RAM < Cache < Registers

Q 1.20: How can cached data have different values in different local caches in SMP?

Example:

1. Memory location `X` contains value `100`
2. CPU_1 reads `X` → its local cache stores `X = 100`
3. CPU_2 reads `X` → its local cache stores `X = 100`
4. CPU_1 updates `X = 200` in its local cache (write-back policy – not yet written to main memory)
5. Now: CPU_1's cache has `X = 200`, CPU_2's cache still has `X = 100`, and main memory may still have `X = 100`
6. Three copies of `X` with **different values** – this is the **cache coherency problem**

Q 1.21: Cache coherency in different processing environments?

a. Single-processor systems:

- Problem arises with **DMA** – device controller may modify memory while CPU cache holds stale data
- Solution: OS invalidates cache entries when DMA transfers modify memory

b. Multiprocessor systems:

- Each CPU has its own local cache; same data may have different values in different caches
- Solution: Hardware cache coherency protocols (e.g., **MESI protocol**, snooping, directory-based)

c. Distributed systems:

- Copies of files/data replicated across multiple nodes for performance and reliability

- When one node updates data, other nodes may have **stale copies**
- Solution: Distributed consistency protocols, file replication protocols, eventual consistency
- Most complex case due to network delays and node failures

Q 1.22: Mechanism for enforcing memory protection?

Base and Limit Register approach:

- Each process is assigned a **base register** (starting address) and **limit register** (size of memory range)
- For every memory access, hardware checks: `base` $\leq$ `address` $<$ `base + limit`
- If the address is **outside** this range $\rightarrow$ hardware generates a **trap** to the OS (segmentation fault)
- Only the OS (in kernel mode) can **modify** the base and limit registers
- This ensures a process **cannot access** another process's memory

Q 1.23: LAN or WAN for each environment?

a. Campus student union $\rightarrow$ **LAN** (single building, small area)
b. Statewide university system $\rightarrow$ **WAN** (multiple locations across a state)
c. A neighborhood $\rightarrow$ **LAN** (small geographic area)

Q 1.24: Challenges of designing OS for mobile devices vs traditional PCs?

- **Limited resources:** Less RAM, slower CPU, limited storage compared to PCs
- **Battery life:** Must aggressively manage power consumption
- **Screen size:** Small displays require different UI designs
- **Touch interface:** OS must support touch, gestures instead of keyboard/mouse
- **Connectivity:** Must handle transitions between WiFi, cellular (3G/4G/5G), Bluetooth
- **Sensors:** Must integrate GPS, accelerometer, gyroscope, proximity sensor
- **Security:** Devices are easily lost/stolen; need remote wipe, encryption
- **App ecosystem:** Must manage app installation, sandboxing, permissions

Q 1.25: Advantages of peer-to-peer systems over client–server systems?

- **No single point of failure:** If one peer goes down, others continue
- **Scalability:** Adding more peers adds more resources (bandwidth, storage)
- **No bottleneck:** Load is distributed; no overloaded central server
- **Cost-effective:** No need for expensive dedicated server hardware
- **Decentralized:** No single entity controls the network

Q 1.26: Distributed applications appropriate for peer-to-peer?

- **File sharing:** BitTorrent, Gnutella
- **Communication:** Skype (older versions), video conferencing
- **Cryptocurrency:** Bitcoin, Ethereum (blockchain)
- **Content delivery:** Distributed CDNs
- **Collaborative computing:** SETI@home, Folding@home (distributed processing)
- **Streaming:** Peer-assisted video streaming

Q 1.27: Advantages and disadvantages of open-source OS?

Advantages:

- **Free to use** – beneficial for students, startups, developing countries
- **Source code available** – developers and students can study, learn, and modify
- **Community-driven** – rapid bug fixes, continuous improvement
- **Customizable** – can be tailored for specific needs
- **Transparency** – security auditable by anyone

Disadvantages:

- **Limited official support** – companies may prefer paid vendor support (disadvantage for enterprises)
- **Steeper learning curve** – disadvantage for non-technical end users
- **Hardware compatibility** – some proprietary drivers not available (disadvantage for gamers/designers)
- **Fragmentation** – too many distributions can confuse new users
- **Security risk** – source code visibility helps attackers find vulnerabilities (disadvantage for security-conscious organizations, though also an advantage for security auditing)

Who benefits: Students, developers, researchers, sysadmins, startups.
Who may find it disadvantageous: Non-technical users, enterprises needing vendor support, gamers needing specific proprietary software/drivers.